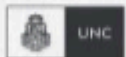


CERTIFICACIÓN UNIVERSITARIA



FCEFYN



Manual para el alumno

Git / Github/ Gitlab




**WE WANT
SKILLS!**

mundosE

Manual para el Alumno:	2
Objetivo del encuentro:	3
Contenido a desarrollar:	
Introducción a Git.....	4
Instalación y Configuración de Git.....	4
Comandos Básicos.....	6
Forks, Pull Requests y Merge Requests.....	9
Bibliografía.....	12

Este manual es un complemento de los encuentros sincrónicos de cada módulo. En él encontraremos un resumen de cada tema tratado en el encuentro, así como también recursos adicionales para poder profundizar sobre los conceptos vistos.

Manual para el Alumno: Git / GitHub / GitLab

Objetivo del encuentro:

El objetivo de este encuentro es que comprendas el uso de **Git** como sistema de control de versiones, su instalación y configuración, el manejo de comandos básicos, y cómo colaborar en equipo a través de **GitHub y GitLab**. Al finalizar, podrás gestionar proyectos de software de manera eficiente, mantener un historial de cambios y participar en flujos de trabajo colaborativos. Además, aprenderás las mejores prácticas para la gestión de código, cómo manejar conflictos en el control de versiones y cómo optimizar tu trabajo en equipo dentro de entornos de desarrollo profesional.

Contenido a desarrollar:

- ✓ **Introducción a Git:** ¿Qué es un sistema de control de versiones y por qué es importante? Comparación con otros sistemas.
- ✓ **Instalación y Configuración:** Cómo instalar y configurar Git en diferentes sistemas operativos. Configuración avanzada y personalización.
- ✓ **Comandos Básicos:** commit, pull, push, merge, branch, stash, reset y rebase.
- ✓ **Trabajo en Equipo:** forks, pull requests y merge requests. Mejores prácticas en la colaboración.
- ✓ **Introducción a GitHub y GitLab:** Creación y gestión de repositorios remotos. Diferencias clave entre ambas plataformas.
- ✓ **Colaboración en Proyectos:** Uso de issues, revisión de código, flujo de trabajo recomendado y estrategias para resolver conflictos de código.

1. Introducción a Git

¿Qué es un Sistema de Control de Versiones?

Un Sistema de Control de Versiones (SCV) permite registrar y administrar los cambios realizados en los archivos de un proyecto a lo largo del tiempo. Su uso es esencial en el desarrollo de software, ya que posibilita la colaboración entre múltiples desarrolladores, manteniendo un historial detallado de todas las modificaciones y permitiendo revertir cambios en caso de errores.

Los SCV pueden clasificarse en:

- Centralizados (CVS, Subversion - SVN): Dependen de un servidor central al que los usuarios deben conectarse para obtener y enviar cambios.
- Distribuidos (Git, Mercurial): Cada usuario posee una copia completa del historial del proyecto, permitiendo trabajar de manera independiente sin necesidad de conexión constante a un servidor.

¿Por qué usar Git?

Git es el sistema de control de versiones más popular y ampliamente utilizado debido a sus múltiples ventajas:

- Es distribuido: Permite a cada usuario tener una copia completa del repositorio, mejorando la seguridad y el acceso al historial de cambios.
- Eficiencia y velocidad: Optimizado para manejar grandes proyectos de manera rápida.
- Excelente manejo de ramas: Facilita el trabajo simultáneo en diferentes características sin afectar la versión estable del código.
- Amplio soporte: Compatible con plataformas como GitHub y GitLab, además de contar con una gran comunidad de soporte.

Recursos para profundizar:

- [Documentación oficial de Git](#)
- [Video: ¿Qué es Git y cómo funciona? - Fazt](#)

2. Instalación y Configuración de Git

Instalación de Git

Para utilizar Git, primero es necesario instalarlo en el sistema operativo correspondiente:

- Windows: Descargar el instalador desde <https://git-scm.com/downloads> y seguir los pasos del asistente.
- MacOS: Instalar con Homebrew ejecutando:

```
brew install git
```

- Linux: Usar el gestor de paquetes de la distribución:

```
sudo apt install git # Debian y Ubuntu  
sudo dnf install git # Fedora y RHEL  
sudo pacman -S git # Arch Linux
```

Configuración Inicial

Una vez instalado Git, es necesario configurarlo con la información del usuario:

```
git config --global user.name "Tu Nombre"  
git config --global user.email "tu@correo.com"
```

Para verificar la configuración actual:

```
git config --list
```

Se pueden definir alias para facilitar el uso de comandos:

```
git config --global alias.st status
```

Esto permite ejecutar `git st` en lugar de `git status`, ahorrando tiempo al escribir comandos.

Recursos para profundizar:

- [Instalación y configuración de Git - Atlassian](#)
- [TUTORIAL de GIT básico. Ep 2 - Instalación y configuración](#)

3. Comandos Básicos

git init

Inicializa un nuevo repositorio en un directorio, convirtiéndolo en un repositorio de Git. Este comando crea una carpeta oculta `.git` donde se almacenará el historial del proyecto.

```
git init
```

git add

Mueve archivos del working directory al staging area, preparándolos para el próximo commit.

```
git add archivo.txt # Agregar un solo archivo
git add . # Agregar todos los archivos modificados
```

El área de preparación permite seleccionar qué cambios se incluirán en el próximo commit.

git commit

Guarda los cambios en el historial del repositorio, junto con un mensaje descriptivo.

```
git commit -m "Mensaje del commit"
```

Cada commit representa un punto en el tiempo al que se puede regresar si es necesario.

git pull

Descarga y fusiona los cambios más recientes del repositorio remoto en la versión local.



```
git pull origin main
```

Es recomendable ejecutar **git pull** antes de comenzar a trabajar para evitar conflictos con cambios realizados por otros colaboradores.

git push

Sube los cambios confirmados al repositorio remoto para que otros desarrolladores puedan acceder a ellos.

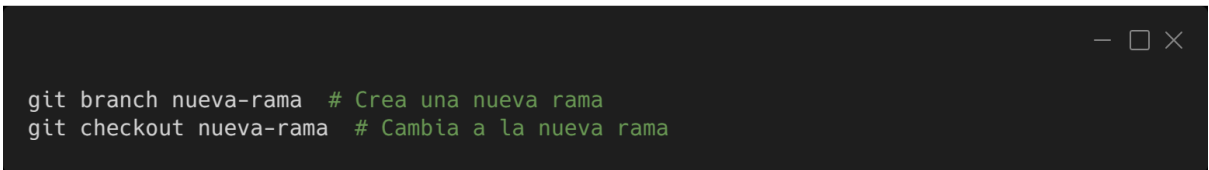


```
git push origin main
```

Este comando hace que los cambios locales sean visibles en la plataforma remota (GitHub o GitLab).

git branch

Administra ramas dentro del repositorio, permitiendo desarrollar nuevas características sin afectar la rama principal.



```
git branch nueva-rama # Crea una nueva rama
git checkout nueva-rama # Cambia a la nueva rama
```

Las ramas facilitan el desarrollo simultáneo y el trabajo en equipo.

git merge

Fusiona los cambios de una rama en otra.

```
git checkout main # Cambia a la rama principal
git merge nueva-rama # Fusiona los cambios de la nueva rama en main
```

Si hay conflictos, Git solicitará resolverlos antes de completar la fusión.

git stash

Guarda temporalmente cambios no confirmados para poder cambiar de rama sin perder el trabajo en progreso.

```
git stash
```

Para restaurar los cambios guardados:

```
git stash pop
```

git reset

Permite deshacer cambios en los commits o en el área de preparación.

```
git reset HEAD~1 # Deshace el último commit manteniendo los cambios en el working directory
```

git rebase

Reorganiza la historia de commits para mantener un historial más limpio.

```
git rebase main
```

Se usa para evitar merge commits innecesarios y mejorar la claridad del historial del proyecto.

Recursos para profundizar:

[12 Comandos de Git que debes de saber](#)

[Git: Comandos Básicos: ATLASSIAN](#)

4. Forks, Pull Requests y Merge Requests:

En el desarrollo colaborativo, es común utilizar **forks**, **pull requests** y **merge requests** para contribuir en proyectos sin afectar la versión principal del código.

- **Fork:** Es una copia independiente de un repositorio en la cuenta de un usuario. Permite realizar modificaciones sin afectar el proyecto original. Es muy utilizado en proyectos de código abierto.
- **Pull Request:** En GitHub, es una solicitud de fusión de cambios de una rama a otra. Permite la revisión del código antes de ser integrado.
- **Merge Request:** En GitLab, funciona de manera similar al pull request, permitiendo la revisión de cambios antes de la fusión en la rama principal.

```
git checkout -b nueva-funcionalidad
git add .
git commit -m "Añadiendo nueva funcionalidad"
git push origin nueva-funcionalidad
```

Recursos para profundizar:

[GitHub - Participando en Proyectos](#)

[Trabajar en equipo con fork y pull request de Github](#)

5. Introduccion a GitHub y GitLab:

GitHub y GitLab son plataformas que permiten alojar repositorios y colaborar en proyectos. Para crear un repositorio:

1. Iniciar sesión en [GitHub](#) o [GitLab](#).
2. Hacer clic en **New Repository**.
3. Asignar un nombre y descripción.

4. Elegir si será público o privado.
5. Copiar la URL del repositorio para clonarlo localmente.

Clonando un repositorio localmente:

```
git clone https://github.com/usuario/repositorio.git
```

Algunas diferencias entre GitHub y GitLab

Si bien existen muchas diferencias entre ambas plataformas acá mencionamos algunas de ellas:

Característica	GitHub	GitLab
Código	Privado con opciones open-source	Open-source con modelo empresarial
CI/CD	GitHub Actions (Integrado nativamente)	Integrado nativamente
Auto Alojamiento	No	Sí
Seguridad	Depende de integraciones	Controles avanzados de acceso
Interfaz	Intuitiva y popular	Completa y personalizable
Comunidad	Muy grande y activa	En crecimiento y con foco en DevOps
Precios	Planes gratuitos y de pago	Planes gratuitos y de pago, con opciones autoalojadas

Recursos para profundizar:

[Introducción a GitLab](#)

[¿Cómo usar GitHub? - ¡Todo lo que necesitás saber!](#)

6. Colaboración en Proyectos:

Uso de Issues y Flujo de Trabajo

Los issues permiten reportar errores, solicitar mejoras o asignar tareas dentro de un proyecto. Permiten gestionar el desarrollo de software de manera organizada y documentada.

Flujo de trabajo con issues:

1. Crear un nuevo issue describiendo el problema o la mejora.
2. Asignar el issue a un colaborador.
3. Crear una rama específica para resolver el issue.
4. Desarrollar la solución y realizar commits vinculados al issue.
5. Crear un pull request o merge request mencionando el issue.
6. Revisar y aprobar los cambios.
7. Cerrar el issue cuando los cambios sean fusionados.

```
git checkout -b fix-issue-45
git add .
git commit -m "Corrigiendo el issue #45"
git push origin fix-issue-45
```

Recursos para profundizar:

[GitLab #7 tareas/issues, en Español](#)

[Domina GitHub: Gestiona Tareas y Proyectos con Issues y Projects](#)

Bibliografía

- **Documentación oficial de Git:** <https://git-scm.com/docs>
- **Guía de GitHub:** <https://docs.github.com/>
- **Guía de GitLab:** <https://docs.gitlab.com/>
- **Libro Pro Git (en español):** <https://git-scm.com/book/es/v2>